Image_Page

เล่มที่ 3 · Backend (Database)

ผู้รับผิดชอบ: ปั้น

ฉบับวันที่ 13 ตุลาคม 2569

1. หน้าที่ของ Backend

Backend คือตัวกลางระหว่างหน้าเว็บกับเครื่อง AI ทุกคำขอจากผู้ใช้ต้องผ่านที่นี่ก่อน หน้าที่หลักมี 4 อย่าง

1. **บัญชีผู้ใช้:** สมัคร, ล็อกอิน, ตรวจ token, สิทธิ์ user / staff
2. **ข้อมูลส่วนตัว:** โน้ตและการตั้งค่า (ภาษา, แท็บ) ของแต่ละบัญชี เก็บใน SQLite
3. **คิวงาน:** รับงานภาพ ตรวจไฟล์ ต่อคิวแยก 2 สาย แล้วส่งต่อให้เครื่อง AI ทีละงาน
4. **Staff Console:** API จัดการผู้ใช้และแดชบอร์ดงานสำหรับเจ้าหน้าที่

หัวข้อ	รายละเอียด
ไฟล์หลัก	`backend/mainBackend.py` (ประมาณ 1,130 บรรทัด)
เทคโนโลยี	FastAPI, SQLite (`Userdata.db`), httpx (เรียกเครื่อง AI), asyncio (คิว)
รันที่	`uvicorn mainBackend:app --host 127.0.0.1 --port 5051` อยู่หลัง nginx พอร์ต 5050
เรียกต่อไปที่	เครื่อง AI `AI_URL` (ค่าเริ่มต้น http://26.84.90.188:8000)

2. การติดตั้งและรัน

```
cd backend
pip install fastapi uvicorn httpx pillow python-multipart
set AI_URL=http://26.84.90.188:8000
uvicorn mainBackend:app --host 127.0.0.1 --port 5051
```

nginx (ไฟล์ `NginxCode.txt`) เปิดพอร์ต 5050 รับเฉพาะ IP เครื่อง frontend (26.213.29.211) จำกัด body 25 MB แล้วส่งต่อ `/api/` ไปที่ 127.0.0.1:5051 พร้อม IP ผู้ใช้ที่ proxy.py ใส่มา ทางอื่นตอบ 404

ค่าที่ตั้งผ่าน env

ตัวแปร	ค่าเริ่มต้น	ความหมาย
AI_URL	http://26.84.90.188:8000	ที่อยู่เครื่อง AI
GEN_CONCURRENCY	1	งานสร้างภาพที่ส่งให้ AI พร้อมกันได้ (ต้องเท่ากับ MAX_CONCURRENT_GEN ฝั่ง AI)
IMG_CONCURRENCY	1	งานแต่งภาพที่ส่งพร้อมกันได้ (ต้องเท่ากับ MAX_CONCURRENT_IMG ฝั่ง AI)
GEN_MAX_WAITING / IMG_MAX_WAITING	10	จำนวนงานที่รอคิวได้ต่อสาย

> ยังไม่มีการสร้างบัญชี staff คนแรกอัตโนมัติ ให้สมัครบัญชีผ่านหน้าเว็บก่อน แล้วตั้งสิทธิ์ใน Userdata.db ด้วยคำสั่ง `UPDATE users SET role='staff' WHERE email='...'` หลังจากนั้นค่อยใช้ Staff Console เลื่อนสิทธิ์คนอื่น

3. ฐานข้อมูล (Userdata.db)

ไฟล์ SQLite ไฟล์เดียว สร้างตารางให้เองตอนเริ่มโปรแกรม (`init_db`) ทุกตารางใช้อีเมลเป็นคีย์
ข้อมูลงานภาพไม่ได้เก็บในฐานข้อมูล เก็บในหน่วยความจำ (หัวข้อ 6)

ตาราง	คอลัมน์	ใช้ทำอะไร
users	email (PK), name, pw, role, created_at	บัญชีผู้ใช้ · pw เป็น hash SHA-256 · role เป็น user หรือ staff
notes	email (PK), content	โน้ตส่วนตัว 1 ชิ้นต่อบัญชี
preferences	email (PK), lang, tabs	ภาษา (ค่าเริ่มต้น th) และรายการแท็บที่แสดง (JSON)

4. การยืนยันตัวตน

- รหัสผ่านเก็บเป็น `sha256(SECRET + รหัสผ่าน)` ต้องยาวอย่างน้อย 8 ตัวอักษร
- token = `base64( {"sub": อีเมล, "exp": เวลาหมดอายุ} | ลายเซ็น HMAC-SHA256 )` อายุ 24 ชั่วโมง
- ทุก endpoint ที่ต้องล็อกอินใช้ `me_user` ตรวจลายเซ็น วันหมดอายุ และต้องยังมีบัญชีใน DB · ไม่ผ่าน → 401
- endpoint ของเจ้าหน้าที่ใช้ `staff_only` ต่อจาก me_user · role ไม่ใช่ staff → 403
- ล็อกอินผิด ไม่ว่าอีเมลไม่มีหรือรหัสผิด ได้ข้อความเดียวกัน (INVALID_CREDENTIALS) กันการไล่เดาอีเมล

> ข้อควรแก้ก่อนใช้งานจริง: SECRET เขียนตายตัวในโค้ดเป็น `dev-secret-change-me` ถ้ามีคนเห็นโค้ด จะปลอม token ได้ ควรย้ายไปอ่านจาก env และควรใช้ hash ที่มี salt แยกรายคน (เช่น bcrypt)

5. รายการ API

ทุก endpoint ขึ้นต้นด้วย `/api/v1` error ทุกตัวตอบรูปแบบเดียวกัน `{"error": {"code", "message", "request_id"}}` และทุกคำตอบมี header `X-Request-Id` ไว้ตามหาใน log

5.1 บัญชีและข้อมูลส่วนตัว

Method	Path	ล็อกอิน	ทำอะไร
POST	/auth/register	—	สมัคร → user + access_token · อีเมลซ้ำ 409 EMAIL_TAKEN · รหัสสั้น 422 WEAK_PASSWORD
POST	/auth/login	—	ล็อกอิน → user + access_token · ผิด 401 INVALID_CREDENTIALS
GET	/auth/me	✓	ข้อมูลบัญชีปัจจุบัน
POST	/auth/logout	✓	ออกจากระบบ (204)

Method	Path	ล็อกอิน	ทำอะไร
GET / PUT	/notes	✓	อ่าน / บันทึกโน้ต
GET / PUT	/preferences	✓	อ่าน / บันทึกภาษาและแท็บ
GET	/health	—	เช็คว่า backend ยังทำงาน

5.2 งานภาพ

ชื่อเครื่องมือ (`{name}`) มี 5 ตัว: `generate`, `remove-background`, `adjust`, `filter`, `blur`

Method	Path	ทำอะไร
POST	/jobs/{name}	ส่งงานเข้าคิว → 202 พร้อม job_id และสถานะแรก (หน้าเว็บใช้ตัวนี้)
GET	/jobs/{id}/events	สถานะสดแบบ SSE ส่งใหม่ทุกครั้งที่สถานะเปลี่ยน + ping ทุก 15 วินาที
GET	/jobs/{id}	สถานะครั้งเดียว (ใช้แทน SSE ได้)
GET	/jobs/{id}/result	ภาพผลลัพธ์ (ยังไม่เสร็จ 409 NOT_READY)
DELETE	/jobs/{id}	ยกเลิกงาน (204)
GET	/queue	ภาพรวมคิวทั้งสองสาย (ไม่ต้องล็อกอิน)
GET	/queue/{id}	ลำดับคิวของงานนี้ + เวลาเฉลี่ย + % (ถ้ากำลังทำ)
POST	/{name}	แบบเก่า: ส่งงานแล้วรอจนได้ภาพในคำขอเดียว (ผ่านคิวเหมือนกัน)

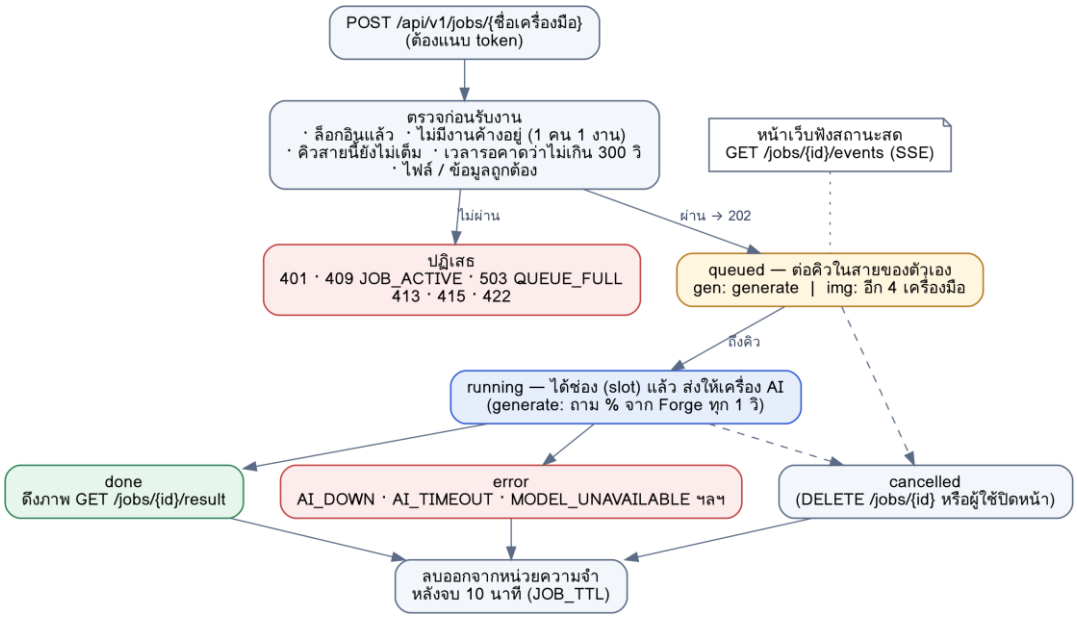
ทุกตัวต้องล็อกอิน ยกเว้น `/queue` · งานของคนอื่นตอบ 404 เหมือนไม่มีงานนั้น

5.3 สำหรับเจ้าหน้าที่ (staff)

Method	Path	ทำอะไร
GET	/admin/jobs	แดชบอร์ด: สรุปสองสาย, งานที่กำลังทำ / รอคิว (ชื่อ อีเมล IP เครื่องมือ สถานะ % เวลา ตัวเลือก), งานที่จบใน 10 นาที
GET	/admin/users	รายชื่อผู้ใช้ทั้งหมด (ไม่มี hash รหัสผ่าน)
PUT / PATCH	/admin/users/{email}/role	เปลี่ยนสิทธิ์ user ↔ staff · ลดสิทธิ์ตัวเอง 409 SELF_DEMOTE
DELETE	/admin/users/{email}	ลบบัญชี พร้อม notes และ preferences · ลบตัวเอง 409 SELF_DELETE

6. ระบบคิวงาน

งานแต่ละชิ้นเป็นออบเจกต์ Job ในหน่วยความจำ (JOBS) มีเจ้าของ เครื่องมือ สาย สถานะ เวลาเริ่ม/จบ และ % ความคืบหน้า งานถูกส่งเข้าสายของตัวเอง แต่ละสายมี semaphore คุมจำนวนที่ส่งให้ AI พร้อมกัน และรายการคิวเรียงตามลำดับเข้า



ภาพที่ 1 · สถานะของงาน 1 ชิ้นตั้งแต่ส่งจนถูกล้าง

6.1 กติกาก่อนรับงาน (_admit)

เงื่อนไข	ไม่ผ่านได้
ผู้ใช้มีงานที่ยังไม่จบอยู่แล้ว (1 คน 1 งาน ทุกสายรวมกัน)	409 JOB_ACTIVE
คิวของสายนั้นเต็ม (กำลังทำ + รอคิว)	503 QUEUE_FULL
เวลารอ + เวลาประมวลผลที่คาดไว้เกิน 300 วินาที	503 QUEUE_FULL
X-Request-ID ซ้ำกับงานที่มีอยู่	409 DUPLICATE_REQUEST

6.2 ตรวจข้อมูลก่อนส่ง AI (_prepare)

เครื่องมือ	ต้องมีไฟล์	ขนาดสูงสุด	ตรวจอะไรเพิ่ม
remove-background	✓	12 MB	ชนิดไฟล์ PNG / JPEG / WebP
adjust, filter, blur	✓	20 MB	ชนิดไฟล์ PNG / JPEG / WebP
generate	—	—	JSON ต้องเป็น object, lora ต้องเป็น none / pixel_art / linear_manga / outline

ไฟล์ผิดชนิดได้ 415 · เกินขนาดได้ 413 · ข้อมูลผิดได้ 422 VALIDATION_ERROR ทั้งหมดถูกปฏิเสธก่อนถึงเครื่อง AI

6.3 ส่งงานให้ AI (_run_job)

- รอจนได้ช่อง (slot) ของสาย → สถานะเป็น running → ส่งต่อทั้งก้อนไปที่ endpoint ของ AI (timeout 300 วินาที)
- งาน generate: ระหว่างรอ ถาม `GET /generate/progress` ของ AI ทุก 1 วินาที เก็บ % (ไม่ถอยหลัง สูงสุด 99) และเวลาที่เหลือ
- สำเร็จ: เก็บภาพไว้ในงาน และบันทึกเวลาประมวลผลไว้คิดค่าเฉลี่ย (ค่ากลางของ 20 ครั้งล่าสุดต่อเครื่องมือ)
- ล้มเหลว: แปลงเป็นรหัสที่หน้าเว็บรู้จัก (ตารางหัวข้อ 7)
- ทุกครั้งที่สถานะเปลี่ยน เรียก `_notify()` ให้ SSE ส่งสถานะใหม่ไปหน้าเว็บทันที
- ผู้ใช้ยกเลิกหรือปิดหน้า → ยกเลิก task และเอาออกจากคิว
- ทุก 60 วินาทีลบงานที่จบเกิน 10 นาที (JOB_TTL) ออกจากหน่วยความจำ

6.4 ข้อมูลความคืบหน้าในสถานะงาน

ฟิลด์	มีเมื่อ	ความหมาย
position, ahead, total	queued	ลำดับคิว (1 = คิวถัดไป), จำนวนงานข้างหน้า, งานทั้งหมดในสาย
progress	running (generate) / done	% จริงจาก Forge 0–99 ระหว่างทำ, 100 เมื่อเสร็จ
eta_sec	running (generate)	เวลาที่เหลือที่ Forge ประเมิน
avg_sec	running (เมื่อมีสถิติแล้ว)	เวลาเฉลี่ยของเครื่องมือนี้ ให้หน้าเว็บประมาณ % เอง
elapsed_sec	running (เฉพาะ /queue/{id})	ทำไปแล้วกี่วินาที (ไม่ใส่ใน SSE เพราะเปลี่ยนทุกเสี้ยววินาที)

7. รหัส error ที่หน้าเว็บได้รับ

HTTP	code	เกิดเมื่อ
401	UNAUTHORIZED	ไม่มี token / token หมดอายุหรือปลอม
403	FORBIDDEN	ไม่ใช่ staff แต่เรียก API ของเจ้าหน้าที่
404	NOT_FOUND	ไม่มีงาน / ไม่ใช่งานของตัวเอง / ไม่มีเครื่องมือนี้
409	JOB_ACTIVE · NOT_READY · DUPLICATE_REQUEST	มีงานค้าง · ภาพยังไม่เสร็จ · รหัสคำขอซ้ำ
413 / 415	FILE_TOO_LARGE · INVALID_FILE_TYPE	ไฟล์ใหญ่เกิน · ชนิดไฟล์ไม่รองรับ
422	VALIDATION_ERROR	ข้อมูลที่ส่งมาไม่ถูกต้อง (เช่น lora ผิด)
503	QUEUE_FULL	คิวเต็ม หรือคาดว่าต้องรอนานเกิน 300 วินาที

HTTP	code	เกิดเมื่อ
502	INVALID_PROMPT / UNPROCESSABLE_IMAGE	AI ตอบ 422 (prompt ผิด / ประมวลผลภาพนี้ไม่ได้)
502	MODEL_UNAVAILABLE · AI_DOWN	Forge ปิด (AI ตอบ 503) · ต่อเครื่อง AI ไม่ติด
504	AI_TIMEOUT	AI ประมวลผลนานเกิน 300 วินาที

ในระบบ /jobs รหัส 3 แถวล่างไม่ได้มาเป็น HTTP status แต่มากับสถานะ error ของงาน (error.code + error.message) ผ่าน SSE หน้าเว็บแสดงข้อความตามรหัส

8. การทดสอบที่เกี่ยวกับ Backend

ไฟล์เทส	สิ่งที่เช็ค
test_register_frontend_backend.py	สมัคร, บันทึกลง DB, รหัสผ่านถูก hash, อีเมลซ้ำ / รหัสสั้นถูกปฏิเสธ
test_auth_and_accounts.py	ล็อกอิน, token ปลอม / หมดอายุ / บัญชีถูกลบ, API staff, ลบผู้ใช้แล้วข้อมูลหาย, Notes / Preferences เป็นของแต่ละคน
test_queue_lanes.py	สองสายไม่ขวางกัน, FIFO, ลำดับคิวแยกสาย, ยกเลิก, ประเมินเวลาแยกสาย
test_job_rules.py	งานของใครของคนนั้น, 1 คน 1 งาน, รหัสคำขอซ้ำ, ล้างงานเก่า
test_progress.py	% จาก Forge ถึงหน้าเว็บผ่าน SSE, Forge ไม่ตอบงานก็ยังเสร็จ
test_admin_jobs.py	แดชบอร์ดเห็นใครทำอะไร, เฉพาะ staff

9. ข้อจำกัดและงานค้าง

- **SECRET ตายตัวในโค้ด** และ hash รหัสผ่านไม่มี salt รายคน ควรแก้ก่อนเปิดใช้จริง
- **งานเก็บในหน่วยความจำ:** restart backend แล้วงานที่รออยู่หาย ผู้ใช้ต้องกดส่งใหม่
- **รายการ lora (และ size ถ้าเพิ่มการตรวจ) ต้องตรงกับฝั่ง AI** เพิ่มค่าใหม่ต้องแก้ทั้งสองที่ มีเทสคอยเช็ค
- **การตรวจขนาดภาพของ generate** แก้แล้วในเครื่อง ยังไม่ commit (ตอนนี้ขนาดผิดได้ข้อความ "ข้อมูล Prompt ไม่ถูกต้อง")
- **nginx `proxy_read_timeout 130s`** สั้นกว่าเวลาสร้างภาพสูงสุด (300 วิ) ไม่กระทบหน้าเว็บที่ใช้ /jobs + SSE (มี ping ทุก 15 วิ) แต่ endpoint แบบเก่า `POST /generate` อาจโดนตัดถ้าสร้างภาพนานเกิน 130 วิ